REALIDAD AUMENTADA COLABORATIVA (RAC)

PRIMERA FASE

Hector Pons, Oscar Almeida, Paz Valdes, Daniel Benito, Mateo Kristich

Índice

1. Visión general del sistema 1
1.1 Descripción 1
1.2 Componentes y topología 1
2. Decisiones de diseño 2
2.1 Hardware 2
2.2 Sistema operativo 2
2.3 Lenguaje y entorno de programación 3
2.4 Diseño de la aplicación 4
2.4.1 Modelo de concurrencia: asyncio (I/O multiplexing) 4
2.4.2 Ventajas de la implementación UDP 4
2.5 Diagrama de clases 6
3. Protocolo de aplicación 7
3.1 Estructura de trama (sin framing) 7
3.2 Tipos de mensajes y esquemas 7
3.3 Ciclo de vida completo 7
3.3.1 Diagrama de secuencia 9
4.Implementación UDP y resultados 10
4.1 Mecanismos clave de la implementación 10
4.1.1 protocol.py 10
4.1.2 server.py 10
4.1.3 client.py 11
4.2 Resultados de los benchmarks 12
5. Planificación cuatrimestral (hitos de evaluación) 13
6. Diagrama de Gantt 14
7. Referencias 15

1. Visión general del sistema

1.1 Descripción

En este informe se explican aspectos importantes acerca de un sistema de simulación de Realidad Aumentado-Colaborativa (RAC), desarrollada con python con “asyncio” y comunicación mediante sockets UDP.

En cada iteración i, un cliente calcula una nueva posición tridimensional, la transmite a sus V − 1 vecinos a través del servidor central, espera la recepción de los V − 1 ACKs de aplicación, mide la latencia de ida y vuelta (RTT) de la iteración y actualiza su promedio acumulado. Los clientes funcionan de forma independiente y asíncrona, por lo que un cliente lento no bloquea al resto. El servidor no retiene mensajes, actúa como un conmutador, se limita a recibir y reenviar los datagramas entre los clientes. No calcula ni almacena mensajes, solo mantiene la tabla de direcciones UDP (ip, puerto) asociada a cada cliente_id. Los clientes se identifican mediante la dirección de orígen de cada datagrama, ya que UDP no dispone de un canal orientado a conexión.

1.2 Componentes y topología

- **1 servidor central UDP:** registra a los clientes por dirección de red, los organiza en grupos y coordina el inicio de la simulación, reenvía los mensajes “POSICION” a los vecinos del grupo y los ACK al cliente que los envió. Al finalizar, recoge las métricas y genera el resumen de resultados.
- **N clientes UDP:** procesos o corrutinas autónomas agrupadas en conjuntos de V vecinos (N divisible por V). Cada cliente mantiene un socket UDP conectado al servidor (remote_addr), lo que permite enviar los datos directamente y aceptar únicamente mensajes procedentes del servidor.
- **Parámetros Principales:**

Símbolo	Significado	Relación
N	Clientes totales	Divisible por V
V	Tamaño de grupo (vecinos)	Cada cliente espera $V - 1$ ACKs por iteración
I	Iteraciones por cliente	Bucle asíncrono e independiente en cada cliente
N / V	Número de grupos	grupo_id = (cliente_id − 1) // V + 1

Topología lógica: estrella centrada en el servidor. Todos los datagramas (registro, posición, ACK, métricas) pasan por el servidor, que conmuta el tráfico entre los miembros del mismo grupo. No existe comunicación directa cliente-a-cliente.

2. Decisiones de diseño

En esta sección se trata la elección de hardware, SO, lenguaje y entorno de programación. También se habla del diseño de la app y se adjunta el diagrama de clases.

2.1 Hardware

El hardware utilizado consta de 6 computadoras:

- **Equipo 1 (Mateo):** ASUS VivoBook X515EA, procesador Intel Core i7-1165G7 (4 núcleos / 8 hilos @ 2.80 GHz), 24 GB RAM, gráfica Intel Iris Xe, SO Ubuntu 24.04 LTS (Linux x86_64).
- **Equipo 2 (Daniel):** GIGABYTE G5 KF, procesador Intel Core i5-12500H (12 núcleos @ 2.50 GHz), 16 GB RAM, gráfica dedicada de la serie G5 / integrada Intel, SO Windows 11 Pro.
- **Equipo 3 (Oscar):** Acer Nitro ANV15-52, procesador Intel Core i5-13420H (8 núcleos @ 2.10 GHz), gráfica de la serie Nitro, SO Windows 11 Home.
- **Equipo 4 (Paz):** Procesador AMD Ryzen 7 5825U (8 núcleos / 16 hilos), 16 GB RAM, gráfica integrada AMD Radeon Graphics.
- **Equipo 5 (Pons):** Apple Mac, procesador Apple M4 Pro, 24 GB de memoria unificada, GPU integrada Apple M4 Pro, SO macOS.
- **Servidor principal:** PC de escritorio, procesador Intel Core i9-9900K (8 núcleos / 16 hilos), 32 GB RAM, gráfica NVIDIA GeForce RTX 2060.

2.2 Sistema operativo

El sistema operativo elegido para correr el servidor es slim Debian una versión reducida del sistema operativo Debian y la imagen de docker utilizada es: python:3.12-slim

Porque se eligió este SO:

- Al ser unix asyncio puede utilizar la llamada al sistema epoll nativamente (muy usado para I/O multiplexing)
- Es un sistema operativo liviano (30MB aprox)

Cabe aclarar que este es el sistema operativo donde corre el servidor, la máquina anfitriona del servidor será el “Servidor principal” y ahí mismo correrán estos clientes en la primera entrega

2.3 Lenguaje y entorno de programación

Como lenguaje de programación se eligió **python** por los siguientes motivos:

- **Sintaxis clara y expresiva:** su legibilidad mejora el mantenimiento del código fuente.
- **Experiencia previa:** dominio y experiencia de parte del equipo con el lenguaje.
- **Soporte para E/S asíncrona (**asyncio**):** librería para multiplexación de entrada/salida, ampliamente utilizado en aplicaciones masivas como FastAPI.

Ecosistema de desarrollo y herramental:

- **Sistema de control de versiones:** Empleo de git y github para el gestionado de versiones.
- **Entorno virtualizado:** Se usa el módulo venv de Python para aislar librerías del proyecto con otros proyectos.
- **Contenedores de software:** Empleo de docker para facilitar el despliegue en las distintas máquinas del equipo.

Librerías empleadas en el proyecto:

- asyncio: librería de multiplexación de E/S asíncrona, permite crear corrutinas para el manejo de envío y recepción de mensajes asíncronos
- socket: librería de sockets de python, usado para agrandar el tamaño de los búferes y evitar descartes de datagramas.
- json: librería estándar de python para el manejo de json, usado en la codificación y decodificación de mensajes.
- time: librería para medir las latencias.
- argparse y sys: librería para procesar parámetros de terminal estándar de python.

Estructura de ficheros del proyecto:

- protocol.py: Definición de protocolo, primitivas de envío, codificación y decodificación.server.py: Servidor central de la app.
- client.py: Cliente que se conecta vía UDP al servidor
- run_test.py: Script que lanza tanto clientes como servidor con los parámetros correspondientes.
- Dockerfile: Dockerfile por el cual se construye la app.

Consideración técnica. Se está analizando la migración del servidor a Go (Golang). La idea es aprovechar la concurrencia del lenguaje usando *Goroutines* y *channels* y evitar el GIL (*Global Interpreter Lock*) de Python.

2.4 Diseño de la aplicación

2.4.1 Modelo de concurrencia: asyncio (I/O multiplexing)

¿Por qué no el modelo hilo por cliente?:

Se descartó utilizar un hilo independiente para cada cliente porque la aplicación está centrada principalmente en operaciones de entrada/salida de red. En este tipo de escenario, Python recomienda el uso de asyncio, ya que está diseñado para ejecutar tareas concurrentes y gestionar comunicaciones de red mediante un modelo asíncrono (Python Software Foundation [PSF], 2026a). El modelo de un hilo por cliente implica mantener un hilo del sistema operativo por cada participante. “asyncio” permite conseguir concurrencia a nivel de tareas sin necesidad de crear un hilo para cada cliente, lo que resulta más adecuado cuando se espera que muchas tareas pasen parte de su tiempo esperando operaciones de red (PSF, 2026b).

El GIL también es un factor a tener en cuenta en CPython. En las versiones que utilizan el GIL, solamente un hilo puede ejecutar código Python a la vez, aunque este bloqueo se libera durante determinadas operaciones de E/S. Por tanto, los hilos siguen siendo válidos para aplicaciones con E/S, pero no proporcionan una ventaja clara frente al modelo asíncrono utilizado en este proyecto (PSF, 2026b, 2026c).

Por estas razones, se opta por asyncio, que permite gestionar los distintos clientes mediante tareas y un event loop, manteniendo la concurrencia necesaria para las comunicaciones UDP sin recurrir a un hilo independiente por cliente.

Elección de E/S asíncrona (asyncio):

- Solo se implementa un hilo que se bloquea/desbloquea a través de la llamada epoll de unix.
- Permite tener miles de sockets no bloqueantes.
- Elimina las condiciones de carrera y permite prescindir de primitivas de sincronización (en código síncrono que no tenga en medio un await).
- Por ser un servidor, es I/O bound, así que la espera de mensajes se mejora con la syscall epoll

2.4.2 Ventajas de la implementación UDP

A. Menos overhead en la capa de transporte:

- **Encabezado:** La cabecera UDP es de **8 bytes**, en cambio, la de TCP es de **20 bytes**. En momentos de alto estrés (muchos clientes) esta reducción se vuelve sustancial.
- **No hay 3-way handshake:** No es necesario el *3-way handshake*, para iniciar la conexión. Esto reduce la cantidad de tráfico en el inicio del sistema

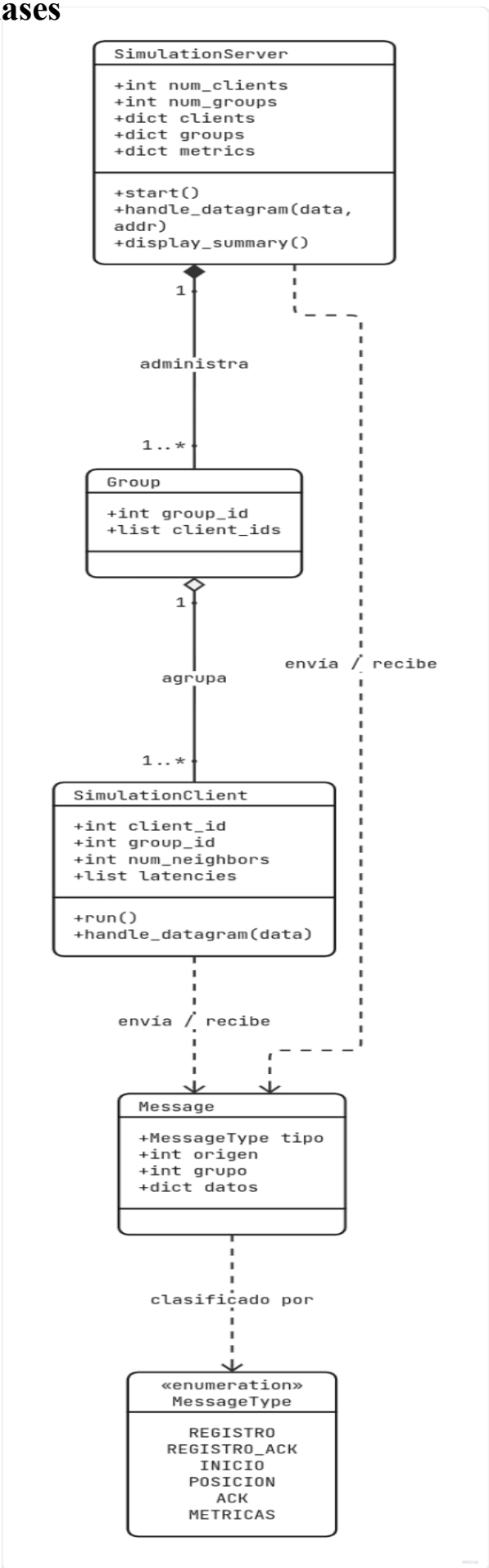
B. No hay necesidad de framing

- **El flujo de TCP:** TCP trata los datos como un flujo continuo de bytes sin fronteras definidas, esto hace que sea necesario implementar un mecanismo de framing a nivel de aplicación (más overhead)
- **Los datagramas de UDP:** Cada datagrama preserva los límites de los mensajes, esto permite prescindir de un mecanismo de framing

C. No hay Head-of-Line Blocking

- **Comportamiento de TCP:** Cuando se pierde un segmento, el receptor descarta los paquetes siguientes que no se correspondan con el número de secuencia esperado. esto genera latencias que pueden llegar a ser críticas con varios clientes funcionando
- **El flujo de los datagramas:** En este escenario donde se necesitan continuamente mensajes de POSICION, los datos viejos pierden interés, así que UDP se ajusta a nuestras necesidades donde no necesitamos un flujo de bytes.

2.5 Diagrama de clases



3. Protocolo de aplicación

3.1 Estructura de trama (sin framing)

En esta versión del proyecto se utiliza UDP, por lo que los datos se envían mediante datagramas independientes. A diferencia de TCP, no es necesario establecer un mecanismo adicional para indicar dónde empieza o termina cada mensaje, ya que UDP mantiene los límites de cada datagrama.

La información que se envía dentro de cada datagrama está organizada en formato JSON y se codifica en UTF-8. De esta forma, cada mensaje contiene directamente los datos necesarios para que el receptor pueda identificarlo y procesarlo.



3.2 Tipos de mensajes y esquemas

Todos los mensajes utilizados por la aplicación tienen una estructura común. Los campos principales son:

- **tipo**: indica qué tipo de mensaje se está enviando.
- **origen**: identifica al cliente que ha generado el mensaje. El valor 0 se reserva para el servidor.
- **grupo**: indica el grupo al que pertenece el cliente.
- **datos**: contiene la información específica de cada tipo de mensaje.

Tipo	Emisor	Destinatario	Descripción
REGISTRO	Cliente	Servidor	Se utiliza para solicitar el identificador y el grupo del client
REGISTRO_ ACK	Servidor	Cliente	Confirma el registro y devuelve el cliente_id y el grupo_id asignados.
INICIO	Servidor	Difusión (broadcast)	Indica que ya se han registrado los N clientes y que puede comenzar la simulación.
POSICION	Cliente	Servidor → vecinos	Contiene la posición del cliente y se envía a los demás miembros de su grupo.
ACK	Vecino	Servidor → cliente	Confirma al cliente original que uno de sus vecinos ha recibido su posición.
METRICAS	Cliente	Servidor	Envía al servidor la latencia media obtenida después de completar las I iteraciones.

3.3 Ciclo de vida completo

El funcionamiento general del proyecto se puede dividir en tres etapas principales

1. **Registro y comienzo de la simulación.**

   Al iniciar, cada cliente envía un mensaje REGISTRO al servidor para solicitar su identificación. El servidor asigna un identificador y un grupo a cada cliente. Cuando se alcanza el número total de clientes N, el servidor envía un mensaje INICIO a todos ellos.

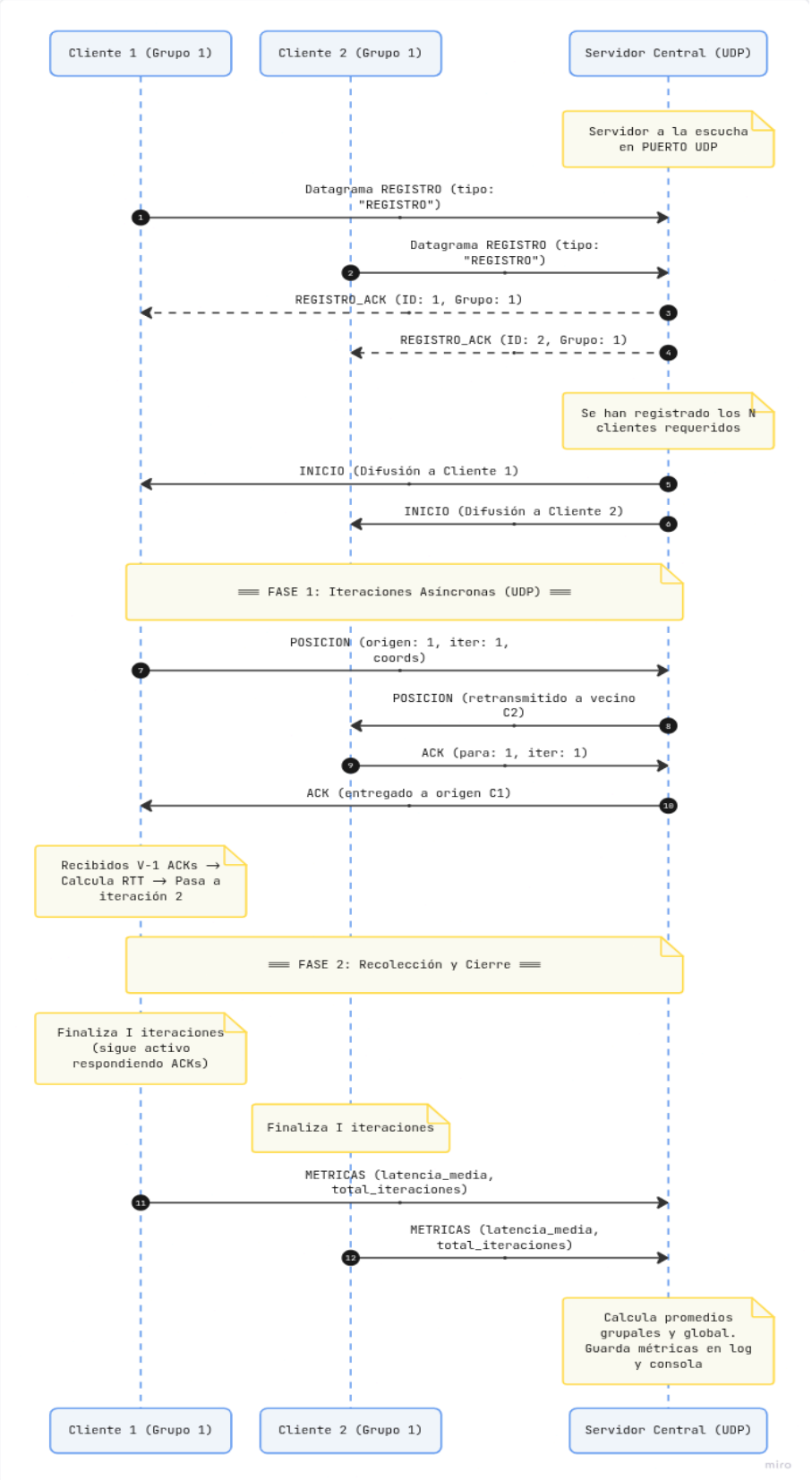
2. **Envío de posiciones y recepción de ACKs.**

   Una vez recibida la señal de inicio, cada cliente comienza sus I iteraciones de forma independiente. En cada iteración genera una nueva posición y la envía al servidor mediante un mensaje POSICION. El servidor reenvía este mensaje a los $V - 1$ vecinos que pertenecen al mismo grupo. Cada vecino responde con un ACK, que vuelve a pasar por el servidor hasta llegar al cliente que envió originalmente la posición. Cuando el cliente recibe todos los ACKs esperados, calcula el RTT de esa iteración y continúa con la siguiente.

3. **Envío de métricas y finalización.**

   Cuando un cliente termina todas sus iteraciones, calcula su latencia media y la envía al servidor mediante un mensaje METRICAS. El cliente permanece activo durante un breve periodo para poder responder a posibles mensajes pendientes de otros vecinos. Por su parte, el servidor recoge las métricas de todos los clientes y, cuando las ha recibido todas, calcula y muestra los resultados finales de la simulación.

3.3.1 Diagrama de secuencia



4.Implementación UDP y resultados

4.1 Mecanismos clave de la implementación

A continuación, se detallan y analizan los mecanismos clave implementados en el sistema de Realidad Aumentada Colaborativa (RAC) mediante UDP, abarcando los módulos de protocolo, servidor, cliente y sus limitaciones conocidas.

4.1.1 protocol.py

Mecanismo	Descripción	Justificación
Serialización de mensajes mediante JSON	Cada mensaje (como POSICION o METRICAS) se serializa a una cadena de texto en formato JSON mediante json.dumps().encode('utf-8') y se deserializa al recibir con json.loads(data.decode('utf-8')).	UDP preserva los límites de los datagramas a nivel de transporte, por lo que cada mensaje viaja de forma independiente y autónoma, eliminando la necesidad de delimitadores o cabeceras de longitud (framing).
Función de envío unificada con dirección opcional (send_message)	Una única función gestiona el envío de tramas: si se proporciona una dirección (addr), utiliza sendto(data, addr) (empleada por el servidor); si es None, transmite directamente al extremo remoto configurado (sendto(data), empleada por el cliente).	Permite reutilizar la lógica de serialización y transmisión tanto en el servidor como en los clientes, adaptándose de forma transparente al rol de cada proceso.

4.1.2 server.py

Mecanismo	Descripción	Justificación
Enrutador centralizado de mensajes (Router único)	Todos los datagramas entrantes de cualquier cliente y tipo son recibidos por un único método central, handle_datagram(), el cual examina el campo tipo y deriva la acción correspondiente (registro, reenvío de posiciones, reenvío de ACKs o métricas).	Centraliza la gestión del tráfico cumpliendo estrictamente con la arquitectura cliente-servidor, donde el servidor actúa como un conmutador puro sin lógica de negocio distribuida.
Identificación de clientes por dirección de red	Dado que UDP es un protocolo sin conexión previa, el servidor identifica a cada participante registrando la tupla de origen (ip, puerto) de cada datagrama, mapeándola mediante un diccionario (addr_to_id) a un identificador único (cliente_id).	Permite asociar de forma unívoca cada mensaje entrante con su respectivo cliente y grupo de trabajo en ausencia de un canal orientado a conexión.

Registro idempotente	Si el servidor recibe un mensaje de registro repetido desde una dirección ya registrada, no crea un nuevo cliente; en su lugar, reasigna o reenvía el mismo identificador existente mediante REGISTRO_ACK sin alterar el estado.	Otorga robustez frente a posibles retransmisiones o duplicaciones de paquetes propias de redes UDP, evitando inconsistencias en la asignación de grupos.
Reenvío optimizado sin recodificación	Al recibir un mensaje de posición, el servidor decodifica el JSON únicamente para identificar el origen y grupo, pero reenvía el datagrama original a los V - 1 vecinos utilizando directamente los bytes crudos recibidos.	Evita la sobrecarga de serializar y codificar el mensaje en JSON repetidas veces para cada vecino, optimizando el rendimiento ante ráfagas de alta concurrencia.
Barrera de sincronización de métricas	El servidor implementa un evento de espera (all_metrics_event) que bloquea el cierre del servidor hasta recibir los reportes de rendimiento (METRICAS) de los N clientes. Una vez completados, ejecuta display_summary().	Garantiza que la fase de cálculo finalice correctamente recopilando y promediando los resultados globales y por grupo tal como exige el enunciado.
Ampliación de buffers de sockets UDP (8 MB)	Se configuran las opciones del socket SO_RCVBUF y SO_SNDBUF a 8 MB mediante setsockopt() durante la creación del transporte.	Mitiga el riesgo de pérdida de paquetes por desbordamiento del búfer del núcleo (kernel) del sistema operativo cuando se reciben ráfagas masivas de datagramas concurrentes.
Manejo defensivo de datagramas inválidos	Si el servidor recibe un paquete con formato corrupto o no decodificable como JSON, la excepción es capturada e ignorada de manera controlada, registrando una advertencia sin detener el servidor.	Asegura la resiliencia del sistema ante anomalías o ruido en la red, impidiendo que un paquete malformado colapse la simulación.

4.1.3 client.py

Mecanismo	Descripción	Justificación
Socket UDP conectado	El cliente asocia su socket UDP al extremo del servidor mediante remote_addr al inicializar el endpoint de datagramas (create_datagram_endpoint).	Simplifica el envío de mensajes permitiendo utilizar sendto(data) sin especificar la dirección del servidor en cada iteración, y filtra por seguridad la recepción exclusiva de paquetes de dicha IP y puerto.
Recepción asíncrona dirigida por eventos	La llegada de datagramas no requiere un bucle de sondeo (polling); asyncio invoca automáticamente el método	Maximiza la eficiencia en el uso de CPU, permitiendo que el cliente permanezca atento a la red de forma reactiva y no bloqueante.

	callback datagram_received() cada vez que un paquete arriba al socket.	
Sincronización mediante banderas y esperas acotadas	Se emplean eventos (asyncio.Event) denominados registered_event y start_event, combinados con asyncio.wait_for(), para pausar la ejecución del cliente hasta recibir los acuses y la señal de inicio.	Evita el consumo innecesario de ciclos de CPU durante las fases iniciales de espera, estableciendo un límite temporal claro de 20 segundos.
Modelo concurrente de rol activo y reactivo	El cliente opera simultáneamente: de forma activa ejecuta sus I iteraciones de envío de posición y espera de ACKs en run(), y de forma reactiva responde instantáneamente con un ACK al recibir la posición de un vecino.	Satisface el requisito de asincronía total de la simulación, permitiendo que cada nodo avance a su propio ritmo sin ralentizar a los demás.
Período de gracia para el cierre coordinado	Tras completar todas sus iteraciones y enviar sus métricas, el cliente ejecuta un tiempo de espera fijo (await asyncio.sleep(1.5)) antes de cerrar su socket.	Proporciona una ventana temporal para atender y responder los últimos acuses de recibo dirigidos a vecinos más lentos, mitigando los timeouts prematuros (aunque se cataloga como aproximado).
Ampliación de buffers de sockets de cliente (4 MB)	Se configuran las opciones SO_RCVBUF y SO_SNDBUF a 4 MB en el socket del cliente mediante setsockopt().	Absorbe de manera eficiente las ráfagas de ACKs concurrentes provenientes de los V-1 vecinos durante cada iteración de la simulación.
Control de tiempo límite (Timeout) de 5 segundos	Las operaciones críticas de espera (recepción de ACKs y mensajes de control) están protegidas por asyncio.wait_for con un límite de 20 segundos, generando un TimeoutError en caso de bloqueo.	Previene que la aplicación quede colgada indefinidamente ante pérdidas de paquetes o caídas de nodos en la red.

4.2 Resultados de los benchmarks

A continuación, se muestran los resultados obtenidos para una configuración de I = 5 iteraciones:

N	*V*	Clientes OK	Errores	Latencia (ms)	Simulación (s)
4	4	4	0	2.925	0.3
25	25	25	0	80.78	1.15
50	25	50	0	166.505	1.53
100	25	100	0	576.8	3,97

Para comprobar cómo afecta el aumento del número de clientes al rendimiento, se han comparado principalmente las configuraciones en las que el tamaño del grupo se mantiene

fijo en V = 25. De esta forma, se puede observar qué ocurre al pasar de N = 25 a N = 50 y posteriormente a N = 100 clientes.

Entre 25 y 50 clientes, la latencia media pasa de 80,78 ms a 166,505 ms, lo que supone aproximadamente duplicar el tiempo al duplicar el número de clientes. En cambio, al pasar de 50 a 100 clientes, la latencia aumenta hasta 576,8 ms, por lo que el crecimiento es bastante mayor que el observado en el caso anterior.

Este comportamiento indica que, a medida que aumenta el número de clientes, el sistema empieza a tener más dificultades para gestionar todos los mensajes. Una posible causa es la carga adicional sobre el event loop y la mayor utilización de CPU, ya que toda la comunicación se gestiona dentro de un único proceso.

La siguiente salida muestra los resultados obtenidos durante la ejecución del benchmark:

```
======================================================
        RESUMEN FINAL DE LA SIMULACIÓN (UDP)
======================================================
 Grupo 1 (Clientes [1, 2, 3, 4]): Latencia Media = 6.579 ms
------------------------------------------------------
 LATENCIA MEDIA GLOBAL (4 clientes): 6.579 ms
======================================================
```

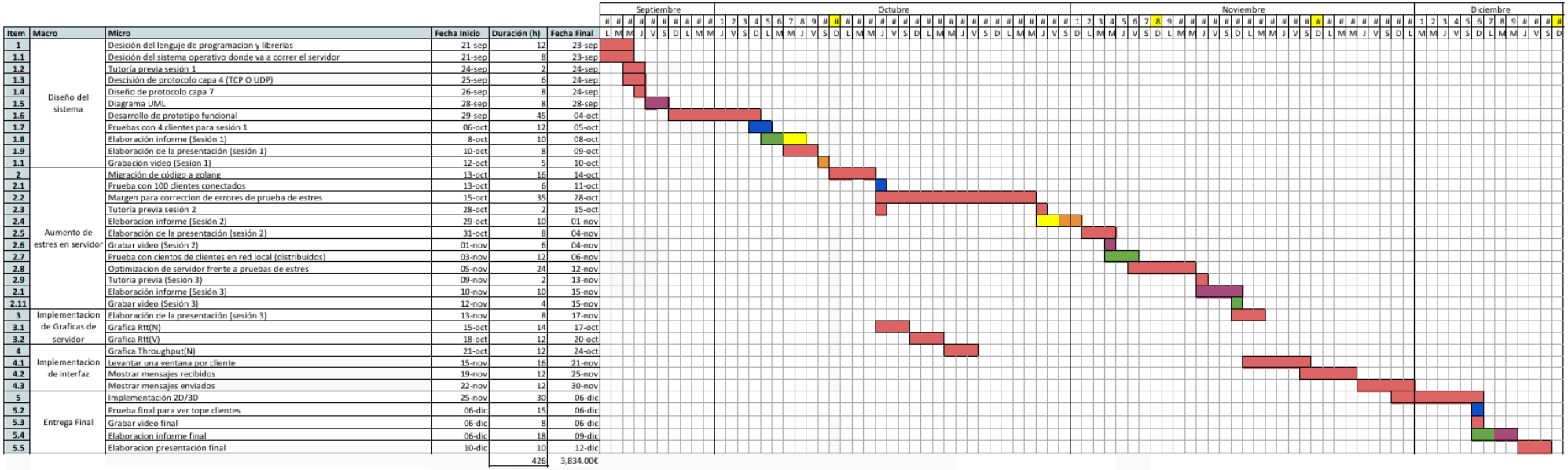
Estos son los puntos de datos que en las Sesiones 2, 3 y 4 se usarán para generar las gráficas de rendimiento requeridas (RTT frente a *N*, RTT frente a *V*).

5. Planificación cuatrimestral (hitos de evaluación)

Hito	Fecha límite	Entregables principales
Sesión 1	11/10/2026 (23:55)	Informe de arquitectura y justificaciones, diagrama de Gantt y prototipo elemental funcional (3-4 clientes, 1 grupo).
Sesión 2	08/11/2026 (23:55)	Ejecución monomáquina escalada a ≥ 100 clientes, análisis de cuellos de botella y código clave.
Sesión 3	22/11/2026 (23:55)	Despliegue distribuido en múltiples máquinas físicas / red local con cientos de clientes y plan de pruebas.
Sesión 4	13/12/2026 (23:55)	Pruebas de límite de escala, gráficas de rendimiento (RTT frente a *N*, RTT frente a *V*, throughput) y memoria final.

6. Diagrama de Gantt

Item	Macro	Micro	Fecha Inicio	Duración (h)	Fecha Final
1	Diseño del sistema	Desición del lenguje de programacion y librerias	21-sep	12	23-sep
1.1		Desición del sistema operativo donde va a correr el servidor	21-sep	8	23-sep
1.2		Tutoría previa sesión 1	24-sep	2	24-sep
1.3		Descisión de protocolo capa 4 (TCP O UDP)	25-sep	6	24-sep
1.4		Diseño de protocolo capa 7	26-sep	8	24-sep
1.5		Diagrama UML	28-sep	8	28-sep
1.6		Desarrollo de prototipo funcional	29-sep	45	04-oct
1.7		Pruebas con 4 clientes para sesión 1	06-oct	12	05-oct
1.8		Elaboración informe (Sesión 1)	8-oct	10	08-oct
1.9		Elaboración de la presentación (sesión 1)	10-oct	8	09-oct
1.1		Grabación video (Sesion 1)	12-oct	5	10-oct
2	Aumento de estres en servidor	Migración de código a golang	13-oct	16	14-oct
2.1		Prueba con 100 clientes conectados	13-oct	6	11-oct
2.2		Margen para correccion de errores de prueba de estres	15-oct	35	28-oct
2.3		Tutoría previa sesión 2	28-oct	2	15-oct
2.4		Eleboracion informe (Sesión 2)	29-oct	10	01-nov
2.5		Elaboración de la presentación (sesión 2)	31-oct	8	04-nov
2.6		Grabar video (Sesión 2)	01-nov	6	04-nov
2.7		Prueba con clientos de clientes en red local (distribuidos)	03-nov	12	06-nov
2.8		Optimizacion de servidor frente a pruebas de estres	05-nov	24	12-nov
2.9		Tutoria previa (Sesión 3)	09-nov	2	13-nov
2.1		Elaboración informe (Sesión 3)	10-nov	10	15-nov
2.11		Grabar video (Sesión 3)	12-nov	4	15-nov
3	Implementacion de Graficas de servidor	Elaboración de la presentación (sesión 3)	13-nov	8	17-nov
3.1		Grafica Rtt(N)	15-oct	14	17-oct
3.2		Grafica Rtt(V)	18-oct	12	20-oct
4	Implementacion de interfaz	Grafica Throughput(N)	21-oct	12	24-oct
4.1		Levantar una ventana por cliente	15-nov	16	21-nov
4.2		Mostrar mensajes recibidos	19-nov	12	25-nov
4.3		Mostrar mensajes enviados	22-nov	12	30-nov
5	Entrega Final	Implementación 2D/3D	25-nov	30	06-dic
5.2		Prueba final para ver tope clientes	06-dic	15	06-dic
5.3		Grabar video final	06-dic	8	06-dic
5.4		Elaboracion informe final	06-dic	18	09-dic
5.5		Elaboracion presentación final	10-dic	10	12-dic
				426	3,834.00€
				IVA	4,639.14€



Daniel	
Héctor	
Oscar	
Mateo	
Paz	
Todos	

7. Referencias

Docker Official Images. (s. f.). *Debian stable-slim*. GitHub. https://github.com/docker-library/repo-info/blob/master/repos/debian/remote/stable-slim.md

Majumder, S. (s. f.). *Async APIs with FastAPI: Patterns, pitfalls & best practices*. Medium. https://shiladityamajumder.medium.com/async-apis-with-fastapi-patterns-pitfalls-best-practices-2d72b2b66f25

Python Software Foundation. (s. f.). *Global interpreter lock (GIL)*. Python Wiki. https://wiki.python.org/moin/GlobalInterpreterLock

Python Software Foundation. (s. f.). *socket — Interfaz de red de bajo nivel*. Python 3.9 documentation. https://docs.python.org/es/3.9/library/socket.html

Python Software Foundation. (2026a). *asyncio — Asynchronous I/O*. Python 3.14.8 documentation. https://docs.python.org/3/library/asyncio.html

Python Software Foundation. (2026b). *Thread states and the global interpreter lock*. Python 3.14.7 documentation. https://docs.python.org/3/c-api/threads.html

Python Software Foundation. (2026c). *threading — Thread-based parallelism*. Python 3.14.7 documentation. https://docs.python.org/3/library/threading.html